```sql
-- B-T00: Consultar disponibilidad inicial de la función
BEGIN;
SELECT * FROM consultar_disponibilidad(1);
ROLLBACK;
```

Data Output | Mensajes | Notificaciones

Showing rows: 1 to 6

	asiento_id integer	estado_actual character varying	reserva_id bigint
1	1	disponible	[null]
2	2	disponible	[null]
3	3	disponible	[null]
4	4	disponible	[null]
5	5	disponible	[null]
6	6	disponible	[null]

```sql
-- Reservar los asientos 1 y 2
BEGIN;
SELECT reservar_asientos(1, 1, 1, ARRAY[1, 2]);
SELECT * FROM consultar_disponibilidad(1);
ROLLBACK;
```

Data Output | Mensajes | Notificaciones

Showing rows

	asiento_id integer	estado_actual character varying	reserva_id bigint
1	1	bloqueado	18
2	2	bloqueado	19
3	3	disponible	[null]
4	4	disponible	[null]
5	5	disponible	[null]
6	6	disponible	[null]

```sql
--B-T01: No se pueden reservar más de 6 asientos
BEGIN;
SELECT reservar_asientos(1, 1, 1, ARRAY[1, 2, 3, 4, 5, 6, 7]);
ROLLBACK;
```

Data Output | Mensajes | Notificaciones

```
ERROR:  No puede seleccionar más de 6 entradas por compra.
CONTEXT:  PL/pgSQL function reservar_asientos(bigint,bigint,bigint,integer[]) line 15 at RAISE

Estado SQL: P0001
```

```sql
--B-T02 Un segundo cliente no puede tomar un asiento ya bloqueado
BEGIN;
SELECT reservar_asientos(1, 1, 1, ARRAY[1]);
SELECT reservar_asientos(1, 1, 1, ARRAY[1]);
ROLLBACK;

```

Data Output | Mensajes | Notificaciones

```
ERROR:  El asiento con ID 1 no se encuentra disponible. Operación cancelada.
CONTEXT:  PL/pgSQL function reservar_asientos(bigint,bigint,bigint,integer[]) line 31 at RAISE

Estado SQL: P0001
```

```sql
--B-T03 Si uno de los asientos falla, ninguno queda reservado
BEGIN;
SELECT reservar_asientos(1, 1, 1, ARRAY[1]);
SELECT reservar_asientos(1, 1, 1, ARRAY[1, 2, 3]);
ROLLBACK;
```

Data Output | Mensajes | Notificaciones

```
ERROR:  El asiento con ID 1 no se encuentra disponible. Operación cancelada.
CONTEXT:  PL/pgSQL function reservar_asientos(bigint,bigint,bigint,integer[]) line 31 at RAISE

Estado SQL: P0001
```

```
37  --B-T04: Una reserva vencida libera su asiento
38  BEGIN;
39  SELECT reservar_asientos(1, 1, 1, ARRAY[1]);
40  UPDATE reserva SET vence_en = now() - interval '1 minute' WHERE estado = 'activa';
41  SELECT liberar_reservas_vencidas();
42  -- Verificar que el asiento volvió a estar disponible
43  SELECT * FROM consultar_disponibilidad(1);
44  ROLLBACK;
```

Data Output | Mensajes | Notificaciones

Showing rows: 1 to 6 | Page No: 1

	asiento_id integer	estado_actual character varying	reserva_id bigint
1	1	disponible	[null]
2	2	disponible	[null]
3	3	disponible	[null]
4	4	disponible	[null]
5	5	disponible	[null]
6	6	disponible	[null]

```
46  --B-T05: Pago rechazado cancela la compra y libera el inventario
47  BEGIN;
48  -- Reservar el asiento 3
49  SELECT reservar_asientos(1, 1, 1, ARRAY[3]);
50  -- Procesar pago como Rechazado
51  SELECT procesar_pago(id_compra, 'Rechazado', 'Tarjeta', 'REF-PAGO-001')
52  FROM compra ORDER BY id_compra DESC LIMIT 1;
53  -- Verificar que el asiento 3 se liberó
54  SELECT * FROM consultar_disponibilidad(1);
55  ROLLBACK;
```

Data Output | Mensajes | Notificaciones

Showing rows: 1 to 6 | Page No:

	asiento_id integer	estado_actual character varying	reserva_id bigint
1	1	disponible	[null]
2	2	disponible	[null]
3	3	disponible	[null]
4	4	disponible	[null]
5	5	disponible	[null]
6	6	disponible	[null]

```sql
--B-T06: Pago aprobado emite boletos correctamente
BEGIN;
-- Reservar el asiento 3
SELECT reservar_asientos(1, 1, 1, ARRAY[3]);
-- Procesar pago aprobado ('Exito')
SELECT procesar_pago(id_compra, 'Exito', 'Tarjeta', 'REF-PAGO-002')
FROM compra ORDER BY id_compra DESC LIMIT 1;
-- Verificar que se crearon los boletos
SELECT * FROM boleto WHERE id_compra = (SELECT id_compra FROM compra ORDER BY id_compra DESC LIMIT 1);
ROLLBACK;

```

Data Output | Mensajes | Notificaciones

Showing rows: 1 to 1 | Page No: 1 of 1

	id_boleto [PK] bigint	id_compra bigint	id_funcion bigint	id_asiento bigint	token_qr uuid	estado character varying (10)	emitido_en timestamp with time zone
1	1	45	1	3	8d4aa164-dfff-4ced-82ad-2965869ad3...	Emitido	2026-10-23 14:36:02.933734

emitido_en timestamp with time zone	usado_en timestamp with time zone
2026-10-23 14:36:02.933734-...	[null]

```sql
--B-T07: Enviar dos veces el mismo pago no duplica nada
BEGIN;
SELECT reservar_asientos(1, 1, 1, ARRAY[1]);
SELECT procesar_pago(id_compra, 'Exito', 'Tarjeta', 'REF-DUP-001')
FROM compra ORDER BY id_compra DESC LIMIT 1; --primer pago
SELECT procesar_pago(id_compra, 'Exito', 'Tarjeta', 'REF-DUP-001')
FROM compra ORDER BY id_compra DESC LIMIT 1; --pago duplicado
ROLLBACK;
```

Data Output | Mensajes | Notificaciones

```
ERROR:  duplicate key value violates unique constraint "pago_referencia_key"
Key (referencia)=(REF-DUP-001) already exists.

Estado SQL: 23505
Detalle: Key (referencia)=(REF-DUP-001) already exists.
Contexto: SQL statement "INSERT INTO pago (id_compra, estado, metodo, referencia, monto)
    VALUES (p_id_compra, v_estado_pago, p_metodo, p_referencia, v_monto)
    RETURNING id_pago"
PL/pgSQL function procesar_pago(bigint,character varying,character varying,character varying) line 26 at SQL statement
```

```sql
--B-T08: Un pago que llega tarde se rechaza y no genera boletos
BEGIN;
SELECT reservar_asientos(1, 1, 1, ARRAY[5]);
UPDATE reserva SET vence_en = now() - interval '5 minutes' WHERE estado = 'activa';
SELECT liberar_reservas_vencidas();
-- Intentar procesar el pago de forma tardía (se procesa como rechazado o cancelado)
SELECT procesar_pago(id_compra, 'Rechazado', 'Tarjeta', 'REF-TARDE-001')
FROM compra ORDER BY id_compra DESC LIMIT 1;
-- Verificar que NO se generaron boletos para esta compra
SELECT * FROM boleto WHERE id_compra = (SELECT id_compra FROM compra ORDER BY id_compra DESC LIMIT 1);
ROLLBACK;
```

Data Output | Mensajes | Notificaciones

id_boleto [PK] bigint	id_compra bigint	id_funcion bigint	id_asiento bigint	token_qr uuid	estado character varying (10)	emitido_en timestamp with time zone	usado_en timestamp with time zone

```sql
--B-T09: Validar el cálculo automático de totales de compra
BEGIN;
SELECT reservar_asientos(1, 1, 1, ARRAY[1, 2, 3]);
-- Consultar la compra y su monto total calculado
SELECT id_compra, total, estado, moneda FROM compra ORDER BY id_compra DESC LIMIT 1;
ROLLBACK;
```

Data Output | Mensajes | Notificaciones

Showing rows: 1 to 1 | Page No: 1 of 1

	id_compra [PK] bigint	total numeric (10,2)	estado character varying (15)	moneda character (3)
1	53	22.50	Pendiente	USD

```sql
-- B-T10 Cada cliente ve solo sus boletos
BEGIN;
SELECT reservar_asientos(1, 1, 1, ARRAY[1]);
SELECT procesar_pago(id_compra, 'Exito', 'Tarjeta', 'REF-PAGO-003')
FROM compra ORDER BY id_compra DESC LIMIT 1;
SELECT * FROM consultar_boletos_usuario(1, 1);
ROLLBACK;
```

Data Output | Mensajes | Notificaciones

Showing rows: 1 to 1 | Page No: 1

	boleto_id bigint	compra_id bigint	funcion_id bigint	asiento_id integer	estado_boleto character varying	qr_token uuid
1	2	46	1	1	Emitido	fd4d6808-598e-4f7c-8967-b5bbcfd86...

```
97
98  -- BT-11: Rembolso y anulación del boleto
99  BEGIN;
100 SELECT reservar_asientos(1, 1, 1, ARRAY[2]);
101 SELECT procesar_pago(id_compra, 'Exito', 'Tarjeta', 'REF-PAGO-004')
102 FROM compra ORDER BY id_compra DESC LIMIT 1;
103 -- Revertir la venta usando la última compra registrada
104 SELECT revertir_venta(id_compra, 1, 'Prueba de reembolso por solicitud')
105 FROM compra ORDER BY id_compra DESC LIMIT 1;
106 -- Verificar disponibilidad e inventario liberado
107 SELECT * FROM consultar_disponibilidad(1);
108 ROLLBACK;
```

Data Output | Mensajes | Notificaciones

Showing rows: 1 to 6 | Page No: 1

	asiento_id integer	estado_actual character varying	reserva_id bigint
1	1	disponible	[null]
2	2	disponible	[null]
3	3	disponible	[null]
4	4	disponible	[null]
5	5	disponible	[null]
6	6	disponible	[null]